⍰ Rust Complete Introduction Guide

A beginner-friendly guide to fast, safe, and modern systems programming.

📘 What is Rust?

Rust is a **systems programming language** focused on:

- Performance ⚡
- Memory safety 🔐
- Concurrency □

It is used for building:

- Operating systems
- Game engines
- Web backends
- High-performance apps

In simple terms:

Rust = C++ performance + Memory safety 🚀

🕰 History of Rust

Year	Event
2010	Rust project started
2015	First stable release
Creator	Graydon Hoare
Developed By	Mozilla

Rust is loved for its **safety and speed**.

⭑ Why Use Rust?

Key Benefits

✓ Memory safety without garbage collection
✓ High performance
✓ Prevents bugs at compile-time
✓ Modern language features
✓ Great for system-level programming

⚡ Core Features of Rust

1. Ownership System

Manages memory safely.

2. Borrowing

Access data without copying.

3. Zero-cost Abstractions

High performance without overhead.

4. Pattern Matching

Powerful control flow.

5. Concurrency Safety

Thread-safe programming.

🛠 Installing Rust

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Verify:

```
rustc --version
```

💡 Basic Example

```
fn main() {
    println!("Hello, Rust!");
}
```

□ Variables

```
let x = 5;
let mut y = 10;
y = 20;
```

🔧 Functions

```
fn add(a: i32, b: i32) -> i32 {
    a + b
}
```

🔄 Control Flow

```
let number = 10;

if number > 5 {
    println!("Greater");
} else {
    println!("Smaller");
}
```

🔃 Loops

```
for i in 1..5 {
    println!("{}", i);
}
```

□ Structs

```
struct Person {
    name: String,
```

```
    age: u32,
}
```

🔗 Ownership Example

```
let s1 = String::from("Hello");
let s2 = s1; // ownership moved
```

□ References & Borrowing

```
fn print_len(s: &String) {
    println!("{}", s.len());
}
```

⚛ Rust in Real Life

Used in:

- System programming 📄
- Web servers 🌐
- Game development 🎮
- Embedded systems ⚙
- Blockchain 🔗

□ Example Program

```
fn greet(name: &str) -> String {
    format!("Hello {}", name)
}

fn main() {
    println!("{}", greet("John"));
}
```

□ Advantages of Rust

✓ Memory-safe
✓ Extremely fast
✓ No runtime errors (most caught at compile time)
✓ Great concurrency support

● Disadvantages of Rust

• Steep learning curve
• Complex ownership system
• Slower compile times

🏆 Best Practices

✓ Understand ownership deeply
✓ Use `cargo` for project management
✓ Write modular code
✓ Handle errors properly
✓ Follow Rust conventions

📌 Quick Summary

Rust helps developers:

- Build fast applications
- Avoid memory bugs
- Write safe concurrent code

✨ Tips to Make Your PDF Look Professional

• Use dark + orange theme □
• Add diagrams for ownership

• Highlight code blocks
• Use fonts like:

- JetBrains Mono
- Inter
- Roboto

Recommended Layout:

- Cover Page
- Table of Contents
- Sections
- Code Examples
- Final Summary